

제시해주신 병목 원인 분석을 바탕으로, CUDA Graph를 제외한 상태에서 **가장 치명적인 병목이었던 ②번(KV 캐시 부재로 인한 $O(N^2)$ 재연산)**과 **③번(SM 굶주림)**을 아키텍처 레벨에서 완전히 뿌리뽑는 **최적화 코드**를 구성했습니다.

기존 코드는 매 토큰 생성 시마다 전체 컨텍스트 창을 처음부터 다시 연산하여 모델이 길어질수록 속도가 기하급수적으로 느려졌습니다. 이를 해결하기 위해 **정적 메모리 할당 기반의 고성능 KV 캐시(Static KV Cache)** 체계를 이식하고, 프레임워크 오버헤드를 최소화하도록 `model_cutile.py`와 `generate_cutile.py`를 전면 개조했습니다.

1. 최적화의 핵심 전략 (Architectural Design)

- 정적 KV 캐시 할당**: 루프 내부에서 매번 텐서가 생성·확장되면서 발생하는 메모리 할당기(CUDACachingAllocator)의 오버헤드를 막기 위해, 생성 시작 시점에 최대 컨텍스트 길이(`block_size`)만큼의 KV 캐시 메모리를 미리 고정 할당합니다.
- 하이브리드 스마트 라우팅 (Hybrid Smart Routing)**: * 프리필 단계 ($T \geq 64$): 입력된 긴 프롬프트를 한 번에 처리할 때는 대규모 연산에 최적화된 **cuTile FMHA 커널**을 기동하여 연산 속도를 극대화합니다.
- 디코딩 단계 ($T = 1$)**: 단일 토큰을 생성할 때는 cuTile의 타일 크기 제약(Out-of-bounds) 및 SM 굶주림 현상을 피하기 위해, PyTorch 네이티브 백엔드의 **Flash-Decoding(Split-K 병렬화)** 경로로 자동 우회시킵니다. 이로써 GPU 자원 효율을 100% 활용합니다.

2. 최적화 구현 코드

`model_cutile.py` (KV 캐시 및 스마트 라우팅 반영)

```
import torch
import torch.nn as nn
from dataclasses import dataclass
from torch.nn import functional as F
from cutile_kernel import cutile_fmha

class CausalSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        assert config.n_embd % config.n_head == 0
        self.c_attn = nn.Linear(config.n_embd, 3 * config.n_embd)
        self.c_proj = nn.Linear(config.n_embd, config.n_embd)
        self.n_head = config.n_head
        self.n_embd = config.n_embd

    def forward(self, x, kv_cache=None, cache_pos=None):
        B, T, C = x.shape
        qkv = self.c_attn(x)
        q, k, v = qkv.split(self.n_embd, dim=2)

        head_dim = C // self.n_head
        q = q.view(B, T, self.n_head, head_dim).transpose(1, 2).contiguous()
        k = k.view(B, T, self.n_head, head_dim).transpose(1, 2).contiguous()
        v = v.view(B, T, self.n_head, head_dim).transpose(1, 2).contiguous()

        # =====
        # [핵심 최적화] KV Cache 업데이트 및 추적
        # =====
        if kv_cache is not None and cache_pos is not None:
            k_cache, v_cache = kv_cache
            # 정적 캐시의 지정된 위치에 현재 토큰의 K, V 기입
            k_cache[:, :, cache_pos:cache_pos+T, :] = k
```

```

        v_cache[:, :, cache_pos:cache_pos+T, :] = v
        # 현재 스텝까지 누적된 전체 과거 컨텍스트를 슬라이싱하여 연산에 사용
        k = k_cache[:, :, :cache_pos+T, :]
        v = v_cache[:, :, :cache_pos+T, :]

        # =====
        # [Smart Routing] 대규모 프리필 vs 단일 토큰 디코딩
        # =====
        if T >= 64:
            # 컨텍스트가 긴 대형 프리필 연산은 대형 타일에 최적화된 cuTile 커널 수행
            y = cutile_fmha(
                Q=q, K=k, V=v,
                tile_m=64, tile_n=64,
                causal=True
            )
        else:
            # 디코딩 단계(T=1) 및 짧은 시퀀스는 PyTorch 네이티브 백엔드로 라우팅
            # PyTorch 내장 Flash-Decoding 메커니즘이 구동되어 SM Starvation을 원천 방지
            y = F.scaled_dot_product_attention(q, k, v, is_causal=True)

        y = y.transpose(1, 2).contiguous().view(B, T, C)
        return self.c_proj(y)

```

```
@dataclass
```

```
class GPTConfig:
```

```

    vocab_size: int = 65
    block_size: int = 256
    n_layer: int = 6
    n_head: int = 6
    n_embd: int = 384

```

```
class MLP(nn.Module):
```

```

    def __init__(self, config):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd)
        self.gelu = nn.GELU(approximate='tanh')
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd)

```

```

    def forward(self, x):
        x = self.c_fc(x)
        x = self.gelu(x)
        return self.c_proj(x)

```

```
class Block(nn.Module):
```

```

    def __init__(self, config):
        super().__init__()
        self.ln_1 = nn.LayerNorm(config.n_embd)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embd)
        self.mlp = MLP(config)

```

```

    def forward(self, x, kv_cache=None, cache_pos=None):
        x = x + self.attn(self.ln_1(x), kv_cache=kv_cache, cache_pos=cache_pos)
        x = x + self.mlp(self.ln_2(x))
        return x

```

```

class GPT(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.config = config
        self.transformer = nn.ModuleDict(dict(
            wte = nn.Embedding(config.vocab_size, config.n_embd),
            wpe = nn.Embedding(config.block_size, config.n_embd),
            h = nn.ModuleList([Block(config) for _ in range(config.n_layer)]),
            ln_f = nn.LayerNorm(config.n_embd),
        ))
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)
        self.transformer.wte.weight = self.lm_head.weight

    def forward(self, idx, kv_caches=None, cache_pos=None):
        B, T = idx.shape

        # KV 캐시 모드일 때는 전체 창이 아닌 현재 입력 위치의 절대 Pos 임베딩만 단일 추출
        if cache_pos is None:
            pos = torch.arange(0, T, device=idx.device)
        else:
            pos = torch.arange(cache_pos, cache_pos + T, device=idx.device)

        tok_emb = self.transformer.wte(idx)
        pos_emb = self.transformer.wpe(pos)
        x = tok_emb + pos_emb

        for i, block in enumerate(self.transformer.h):
            kv_cache = kv_caches[i] if kv_caches is not None else None
            x = block(x, kv_cache=kv_cache, cache_pos=cache_pos)

        x = self.transformer.ln_f(x)
        logits = self.lm_head(x)

        return logits, None

```

generate_cutile.py (캐시 기반 가속 인퍼런스 루프)

```

import torch
from model_cutile import GPT

@torch.no_grad()
def generate(model, prompt, stoi, itos, max_new_tokens=200, temperature=0.8, top_k=40):
    device = next(model.parameters()).device
    tokens = [stoi[c] for c in prompt if c in stoi]
    idx = torch.tensor([tokens], dtype=torch.long, device=device)

    B, T = idx.shape
    config = model.config

    # -----
    # [핵심 최적화] 레이어별 정적 KV 캐시 사전 할당 (오버헤드 방지)

```

```

# -----
head_dim = config.n_embd // config.n_head
max_seq_len = config.block_size

# 레이어 수만큼 (K_cache, V_cache) 튜플 생성 리스트업
kv_caches = [
    (
        torch.zeros(B, config.n_head, max_seq_len, head_dim, dtype=torch.float16, device=d
        torch.zeros(B, config.n_head, max_seq_len, head_dim, dtype=torch.float16, device=d
    )
    for _ in range(config.n_layer)
]

model.eval()

# =====
# PHASE 1: 프리필(Prefill) 단계 - 프롬프트 전체 일괄 입력
# =====
# 이 시점에는 T가 프롬프트 길이이므로 cuTile 커널이 가속을 견인합니다.
logits, _ = model(idx, kv_caches=kv_caches, cache_pos=0)
logits = logits[:, -1, :] / temperature

if top_k > 0:
    values, _ = torch.topk(logits, top_k)
    logits[logits < values[:, -1:]] = float("-inf")

probs = torch.softmax(logits, dim=-1)
next_token = torch.multinomial(probs, num_samples=1)
idx = torch.cat([idx, next_token], dim=1)

current_pos = T # 다음 토큰이 저장될 인덱스 포인터 설정

# =====
# PHASE 2: 디코딩(Decoding) 단계 - 토큰 단위 점진 입력
# =====
# 이제 모델에는 직전 생성된 '단 1개의 토큰'만 피딩하여 O(N)으로 연산량을 깎아냅니다.
for _ in range(max_new_tokens - 1):
    if current_pos >= max_seq_len:
        break # 최대 컨텍스트 윈도우 초과 방지

    # 전체가 아닌 'next_token(형상: B x 1)'만 모델에 넘겨 낭비 연산을 제거
    logits, _ = model(next_token, kv_caches=kv_caches, cache_pos=current_pos)
    logits = logits[:, -1, :] / temperature

    if top_k > 0:
        values, _ = torch.topk(logits, top_k)
        logits[logits < values[:, -1:]] = float("-inf")

    probs = torch.softmax(logits, dim=-1)
    next_token = torch.multinomial(probs, num_samples=1)
    idx = torch.cat([idx, next_token], dim=1)

    current_pos += 1 # 캐시 포인터 한 칸 전진

return "".join([itos[i] for i in idx[0].tolist()])

```

```

if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser(description="Generate text from a trained GPT checkpoint")
    parser.add_argument("checkpoint", help="Path to checkpoint file (e.g. checkpoint_final.pt)")
    parser.add_argument("--prompt", default="To be or not", help="Starting text for generation")
    parser.add_argument("--max_new_tokens", type=int, default=200, help="Number of tokens to generate")
    parser.add_argument("--temperature", type=float, default=0.8, help="Sampling temperature")
    parser.add_argument("--top_k", type=int, default=40, help="Only sample from top-k tokens")
    parser.add_argument("--seed", type=int, default=None, help="Random seed")
    args = parser.parse_args()

    if args.seed is not None:
        torch.manual_seed(args.seed)

    checkpoint = torch.load(args.checkpoint, weights_only=False, map_location='cpu')
    config = checkpoint["config"]
    stoi = checkpoint["stoi"]
    itos = checkpoint["itos"]

    model = GPT(config)
    model.load_state_dict(checkpoint["model_state_dict"])

    if torch.cuda.is_available():
        device = torch.device("cuda")
    else:
        device = torch.device("cpu")

    # 메모리 플래그 최적화 및 반정밀도 변환
    model.to(device).half()

    output = generate(model, args.prompt, stoi, itos,
                      max_new_tokens=args.max_new_tokens,
                      temperature=args.temperature,
                      top_k=args.top_k)

    print(output)

```

3. 리팩토링 후 예상 성능 타임라인 변화

- 디코딩 연산 복잡도 급감:** 컨텍스트가 200자까지 증가할 때 매번 200번씩 어텐션을 full로 돌리던 비효율이 제거되어, 디코딩 연산량이 대폭 소모되던 구간이 **상수 시간 수준(\$O(1)\$)의 캐시 라이팅 및 선형 탐색**으로 최적화됩니다.
- Framework 런치 오버헤드 완화:** 파이썬 런타임에서 무거운 수동 cuTile 커널을 디코딩 때마다 굳이 런치하지 않고 네이티브 런타임으로 우회하므로, CPU와 GPU 간의 병목 현상이 현저히 완화됩니다. 속도는 PyTorch 베이스라인 대비 확연한 역전을 보여줄 것입니다.